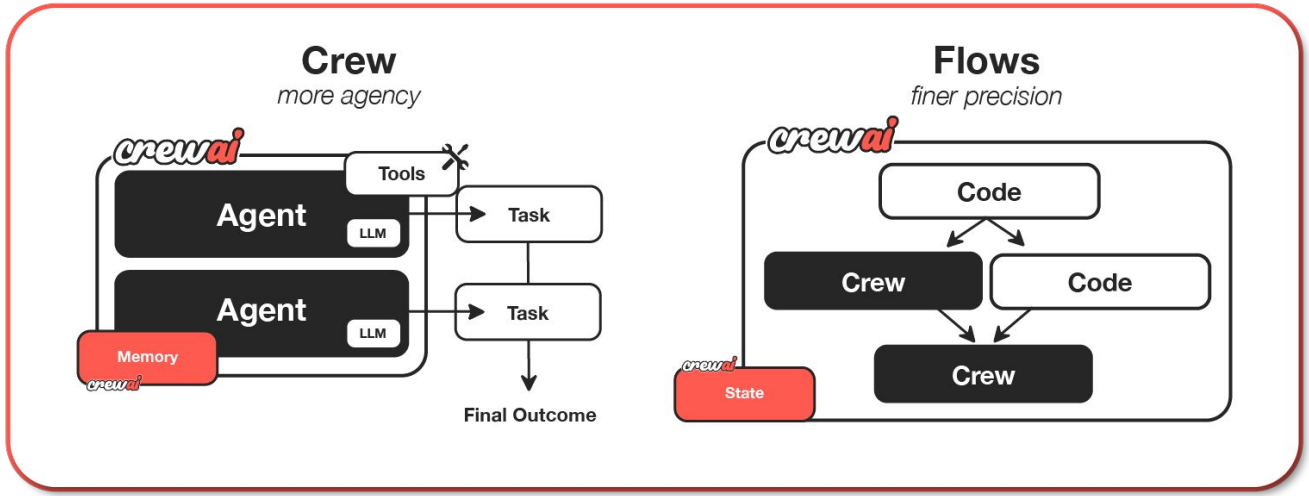
fwdays academy

Модуль 5 .2

CrewAI

CrewAI

CrewAI — це легкий, високошвидкісний фреймворк на Python для координації команд автономних AI-агентів. Він створений «з нуля» і не залежить від LangChain чи інших агентських фреймворків. За своєю архітектурою CrewAI наслідує структуру організації: існує Crew (команда), що складається з агентів, і виконує певний процес, що розбивається на індивідуальні завдання.

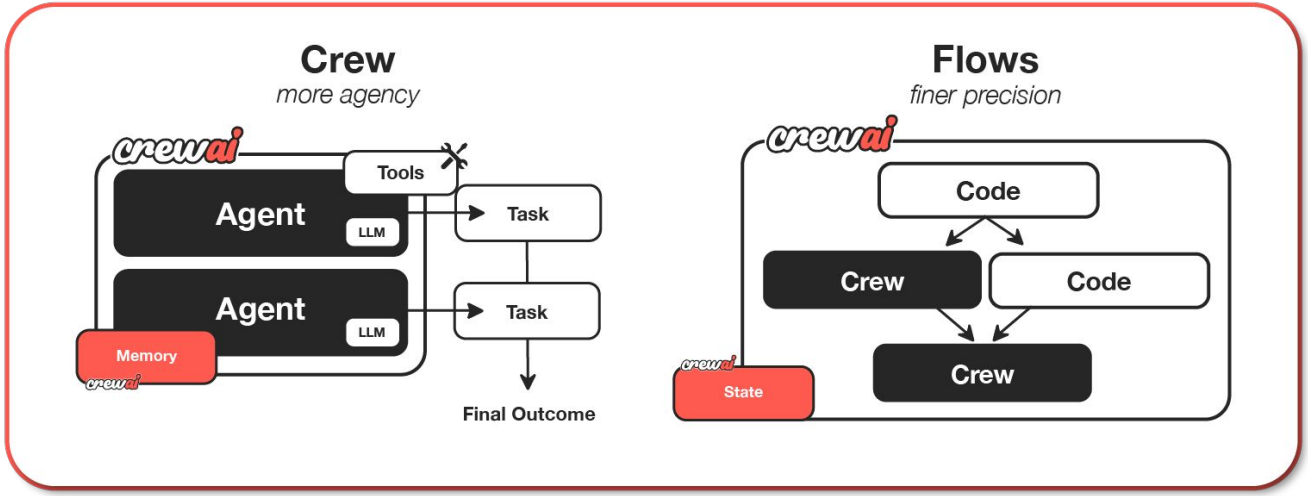


Реклама

CrewAI розкриває справжній потенціал багатоагентної автоматизації, забезпечуючи найкраще у своєму класі поєднання швидкості, гнучкості та контролю за допомогою **Crews of AI Agents** або **Flows of Events**:

- **Автономний фреймворк** : створений з нуля, незалежно від LangChain або будь-якого іншого фреймворку агентів.
- **Висока продуктивність** : оптимізовано для швидкості та мінімального використання ресурсів, що забезпечує швидше виконання.
- **Гнучке налаштування низького рівня** : повна свобода налаштування як на високому, так і на низькому рівнях — від загальних робочих процесів та архітектури системи до детальної поведінки агентів, внутрішніх підказок та логіки виконання.
- **Ідеально підходить для будь-якого випадку використання** : доведена ефективність як для простих завдань, так і для дуже складних реальних сценаріїв корпоративного рівня.
- **Потужна спільнота** : Підтримується швидкозростаючою спільнотою з понад **100 000 сертифікованих розробників**, які пропонують комплексну підтримку та ресурси.

Crew vs Flows



Оберіть **Crews**, коли: Вам потрібне автономне вирішення проблем, творча співпраця або дослідницькі завдання

Оберіть **Flows**, коли: вам потрібні детерміновані результати, можливість аудиту або точний контроль над виконанням

Чому CrewAI?

🧠 **Автономна робота** : Агенти приймають розумні рішення на основі своїх ролей та доступних інструментів
📝 **Природна взаємодія** : Агенти спілкуються та співпрацюють як члени людської команди
🛠️ **Розширюваний дизайн** : легко додавати нові інструменти, ролі та можливості
🚀 **Готовий до production** : створений для надійності та масштабованості в реальних застосуваннях
🔒 **Орієнтований на безпеку** : Розроблено з урахуванням вимог безпеки підприємства
💰 **Економічно ефективний** : оптимізовано для мінімізації використання токенів та викликів API

Easy come, easy go

```
crewai create crew <your_project_name>
```

```
my_project/
├── .gitignore
├── knowledge/
├── pyproject.toml
├── README.md
├── .env
└── src/
    └── my_project/
        ├── __init__.py
        ├── main.py
        ├── crew.py
        ├── tools/
        │   ├── custom_tool.py
        │   └── __init__.py
        └── config/
            ├── agents.yaml
            └── tasks.yaml
```

Файл	Мета
`agents.yaml`	Визначте своїх агентів ШІ та їхні ролі
`tasks.yaml`	Налаштування завдань та робочих процесів агента
`.env`	Зберігайте ключі API та змінні середовища
`main.py`	Точка входу в проект та процес його виконання
`crew.py`	Оркестрування та координація екіпажу
`tools/`	Каталог інструментів для користувацьких агентів
`knowledge/`	Довідник для бази знань

Створення ефективних агентів 1/2

Правило 80/20: зосередьтеся на завданнях, а не на агентах

80% ваших зусиль має бути спрямовано на проектування завдань, і лише 20% – на визначення агентів.

Чому? найдосконаліший агент зазнає невдачі з погано розробленими завданнями, але добре розроблені завдання можуть покращити навіть простий агент.

Це означає:

- Витрачайте більшу частину свого часу на написання **чітких інструкцій** до виконання **завдань**
- Визначте **детальні вхідні дані** та **очікувані результати**
- Додайте приклади та **контекст для керівництва виконанням**
- Присвятіть решту часу **ролі**, **меті** та **передісторії агента**

Створення ефективних агентів 2/2

Основні принципи ефективного дизайну агентів

1. Структура «Роль-Ціль-Передумови»
2. Спеціалісти перевага над універсалами
3. Балансування спеціалізації та універсальності
4. Встановлення відповідних рівнів експертизи

https://docs.crewai.com/en/guides/agents/crafting-effective-agents

Flows

Рекомендації:

1. **Поєднуйте різні моделі взаємодії зі штучним інтелектом** – використовуйте команди для складних спільних завдань, прямі виклики LLM для простіших операцій та звичайний код для процедурної логіки.
2. **Створення подієво-керованих систем** – визначення того, як компоненти реагують на певні події та зміни даних
3. **Зберігайте стан між компонентами** – обмінюйтеся даними та перетворюйте їх між різними частинами вашої програми.
4. **Інтеграція із зовнішніми системами** – безперешкодно підключайте свій робочий процес зі штучним інтелектом до баз даних, API та інтерфейсів користувача
5. **Створення складних шляхів виконання** — проектування умовних розгалужень, паралельної обробки та динамічних робочих процесів

https://docs.crewai.com/en/guides/agents/crafting-effective-agents

MCP

```python
from crewai import Agent

agent = Agent(
    role="Research Analyst",
    goal="Research and analyze information",
    backstory="Expert researcher with access to external tools",
    mcps=[
        "https://mcp.exa.ai/mcp?api_key=your_key",          # External MCP server
        "https://api.weather.com/mcp#get_forecast",         # Specific tool from server
        "crewai-amp:financial-data",                        # CrewAI AMP marketplace
        "crewai-amp:research-tools#pubmed_search"           # Specific AMP tool
    ]
)
```

RAG

```python
from crewai_tools import RagTool

# Create a RAG tool with default settings
rag_tool = RagTool()

# Add content from a file
rag_tool.add(data_type="file", path="path/to/your/document.pdf")

# Add content from a web page
rag_tool.add(data_type="web_page", url="https://example.com")

# Define an agent with the RagTool
@agent
def knowledge_expert(self) -> Agent:
    '''
    This agent uses the RagTool to answer questions about the knowledge base.
    '''
    return Agent(
        config=self.agents_config["knowledge_expert"],
        allow_delegatic
```

Практика

https://github.com/agentspro/module5